# DATA 211 – Introduction to Data Science
# Topic 6: Linear Regression

Question bank: 40 multiple choice + 10 programming

Questions from DATA 211 / ISE 291 old exams and quizzes (AI_Work). **[new]** marks new questions written in the same style.

## Part A: Multiple choice

### Q1.

P2 (Final practice), A-2

_____ describes the strength and direction of the relationship between variables, whereas _____ models the relationship between variables of interest.

(A) Gradient Descent, Regression

(B) Penalty Function, Correlation

(C) Correlation, Cross Validation

(D) Correlation, Regression

**Answer:** (D). Correlation measures strength/direction; regression models how the inputs relate to the output.

My answer

### Q2.

Final Exam T241, Q14

What does the correlation coefficient $r$ indicate in the context of linear regression?

(A) The intercept of the regression line

(B) The predicted slope of the regression line

(C) The strength and direction of a linear relationship between two variables

(D) It determines which variable is the dependent variable

**Answer:** (C). $r$ measures strength and direction; it is used in $\beta_1 = r\, sd_y/sd_x$ but is not the slope itself.

My answer

### Q3.

P5 (Final practice), Q5

Which of the following is False about linear regression?

(A) It is used for prediction.

(B) It is an unsupervised learning model.

(C) It defines a relationship between independent and dependent variables.

(D) It defines the dependent variable as a linear function of independent variables.

**Answer:** (B). Regression learns from data with a known output $y$, so it is supervised.

My answer

### Q4.

P8 (Quiz 03), Q10

A regression model stated by $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$ is _____ regression.

(A) Single input, single output linear.

(B) Two inputs, single output nonlinear.

(C) Three inputs, single output linear.

(D) Two inputs, single output linear.

**Answer:** (D). Two inputs $X_1, X_2$, one output $Y$, linear in the coefficients (MISO).

My answer

### Q5.

P5 (Final practice), Q6

In a single input, single output regression $y = \beta_0 + \beta_1 x$, the parameters $(\beta_0, \beta_1)$ refer to

(A) ($y$-intercept, slope)

(B) ($x$-intercept, MSE)

(C) ($x$-intercept, correlation)

(D) (pca_1, pca_2)

**Answer:** (A). $\beta_0$ is the value of $\hat{y}$ at $x = 0$; $\beta_1$ is the slope.

My answer

### Q6.

P5 (Final practice), Q7

The least squares regression is $y = -0.2 + 0.03x$, where $y$ is blood pressure and $x$ is the amount of salt consumed. This implies that

(A) one unit salt increase leads to an average pressure decrease of 20%

(B) one unit salt increase leads to an average pressure decrease of 17%

(C) one unit salt increase leads to an average pressure increase of 17%

(D) one unit salt increase leads to an average pressure increase of 3%

**Answer:** (D). The slope $+0.03$ is the average change in $y$ per one-unit increase in $x$.

My answer

**Q7.** Final-213 Solved, Q24

In a SISO linear regression model, if we change the input variable by 1 unit, how much will the output variable change?

(A) by 1

(B) no change

(C) by $\beta_0$

(D) by $\beta_1$

**Answer:** (D). The slope $\beta_1$ is the change in $\hat{y}$ for each one-unit increase in $x$.

My answer

**Q8.** P5 (Final practice), Q8

Given that the correlation of the input variable and output variable is positive, then the regression equation has

(A) a positive intercept.

(B) a negative intercept.

(C) a positive slope.

(D) a negative slope.

**Answer:** (C). $\beta_1 = r\, sd_y/sd_x$ and standard deviations are positive, so $\beta_1$ has the sign of $r$.

My answer

**Q9.** Final-213 Solved, Q3

The slope of the regression line of $Y$ on $X$ is also referred to as the:

(A) Regression coefficient of $X$ on $Y$

(B) Correlation coefficient of $X$ on $Y$

(C) Regression coefficient of $Y$ on $X$

(D) Correlation coefficient of $Y$ on $X$

**Answer:** (C). The slope $\beta_1$ of the line predicting $Y$ from $X$ is the regression coefficient of $Y$ on $X$ (old key said A).

My answer

**Q10.** Quiz 3 T253 (Sec 04&05), Q6

In the case of SISO, the closed form solutions of parameters $B_1$ and $B_0$ are _____ and _____ respectively.

(A) $r \cdot (STD_x/STD_y),\ \bar{y} - B_1\bar{x}$

(B) $\bar{y} - B_0\bar{x},\ r \cdot (STD_y/STD_x)$

(C) $r \cdot (STD_y/STD_x),\ \bar{y} - B_1\bar{x}$

(D) $\bar{y} - B_0\bar{x},\ r \cdot (STD_x/STD_y)$

**Answer:** (C). Slope $\beta_1 = r\, sd_y/sd_x$; intercept $\beta_0 = \bar{y} - \beta_1\bar{x}$.

My answer

**Q11.** P2 (Final practice), B-7

Assume you have an input variable denoted as $X$, and an output variable denoted as $y$. Given the mean of $Y = 124$ and the mean of $X = 44$, standard deviation of $Y = 14$ and the standard deviation of $X = 4.5$, and the Pearson's $r$ correlation $= 0.92$. What is the estimated value of $Y$ for a new observation $X = 27$?

(A) 75.34

(B) $-1.94$

(C) 2.86

(D) $-49.46$

**Answer:** (A). $\beta_1 = 0.92(14/4.5) = 2.862$, $\beta_0 = 124 - 2.862(44) = -1.938$, $\hat{y} = -1.938 + 2.862(27) = 75.34$.

My answer

**Q12.** Final Exam T241, Q20

What is the output of the following code (without calculation) for fitting a linear regression model using scikit-learn?

```
df = pd.DataFrame({'x':[1,2,3], 'y':[2,4,6]})
model = LinearRegression()
model.fit(df[['x']], df['y'])
print(model.coef_)
```

(A) 2.0

(B) 0.0

(C) 0.5

(D) 1.0

**Answer:** (A). The points lie exactly on $y = 2x$, so the slope is 2 (prints `[2.]`).

My answer

**Q13.** Final Exam T241, Q21

What is the Python code to predict the score of a new student with attitude 74 using the previously estimated regression model (named as `model`)?

(A)
```
new_data = np.array([[74]])
predicted_score = model.predict(new_data)
print(predicted_score)
```

(B)
```
predicted_score = model.fit([[74]], model.coef_)
print(predicted_score)
```

(C)
```
predicted_score = model.predict(74)
print(predicted_score)
```

(D)
```
new_data = model.transform([[74]])
predicted_score = new_data[0]
print(predicted_score)
```

**Answer:** (A). `predict` needs a 2-D input (one row, one column); a bare scalar raises an error, `fit` re-trains, and `LinearRegression` has no `transform`.

My answer

**Q14.** P7 (Final practice), Q9

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression(fit_intercept=True)
X = reg.coef_[0]
Y = reg.intercept_
```

From the above code in LinearRegression, the X and Y are _____

(A) $\beta_0$ and $\beta_1$, respectively

(B) $\beta_1$ and $\beta_0$, respectively

(C) Input and output variables

(D) Hyperparameters

**Answer:** (B). `coef_[0]` is the slope of the first input; `intercept_` is $\beta_0$.

My answer

**Q15.** Topic 7 practice solution, Q10

The `fit_intercept=False` command in `LinearRegression` gives

(A) $y = 0$

(B) slope equal to zero

(C) $y$-intercept equal to zero

(D) $x$-intercept equal to zero

**Answer:** (C). The line is forced through the origin: $\beta_0 = 0$.

My answer

### Q16.

P6 (Final practice), Q6

What does the Ordinary Least Square (OLS) seek to minimize in linear regression?

(A) Mean of the errors

(B) Summation of the absolute errors

(C) Mean of the square of errors

(D) Maximum error value

**Answer:** (C). OLS minimizes the squared error function (mean squared error).

My answer

### Q17.

ISE291 Final Practice Questions, Q17

Given 5 data points in a linear regression model where the residual errors ($e = y - y_{pred}$) are 1, 3, 1, $-3$, and $-2$. Calculate the Mean Squared Error (MSE).

(A) Zero

(B) 4.8

(C) 24

(D) 1

**Answer:** (B). $(1 + 9 + 1 + 9 + 4)/5 = 24/5 = 4.8$.

My answer

### Q18.

P1 (Quiz 3 T251), Q1

The Gradient Descent method for Linear Regression is

(A) A machine learning algorithm for the recognition of classes

(B) An algorithm for finding the minimum of an error function

(C) A method for calculating the correlation coefficient

(D) A technique for feature selection in regression models

**Answer:** (B). It walks downhill on the error function to its lowest point.

My answer

### Q19.

P4 (Final practice), Q8

The value of the step size $\lambda$, used in the gradient descent algorithm, should be small so that ...

(A) The algorithm becomes more robust to noise.

(B) The convergence of the algorithm becomes faster.

(C) The algorithm does not miss/overshoot the value of minimum.

(D) The cost function becomes linear.

**Answer:** (C). A large step can overshoot the bottom of the error bowl (small steps are actually slower).

My answer

### Q20.

P2 (Final practice), A-6

Which of the following statements is FALSE about the Gradient Descent Algorithm (GDA)?

(A) The first step in the GDA is to start with random values of $\beta_0$ and $\beta_1$ and calculate the resulting error

(B) The Gradient Descent Algorithm finds the best values of $\beta_0$ and $\beta_1$ that minimize the Squared Error Function

(C) The terminating criterion for GDA is when $\varepsilon_{new} \leq \varepsilon_{old}$ (the new error is less than the old error)

(D) The GDA can be used to solve penalized linear regression (Ridge and Lasso) as well as OLS

**Answer:** (C). GDA keeps going while the error falls; it stops when the error no longer falls ($\varepsilon_{new} \geq \varepsilon_{old}$).

My answer

### Q21.

P8 (Quiz 03), Q7

"The gradient descent algorithm requires the squared error function to be convex and differentiable." Which statement about this claim is correct?

(A) True: the gradient must exist, and a convex (bowl-shaped) error guarantees that walking downhill reaches the single lowest point.

(B) False: gradient descent requires the error function to be linear.

(C) False: gradient descent only works when a closed-form solution does not exist.

(D) False: gradient descent requires the error to reach exactly zero.

**Answer:** (A). Original was True/False with answer True.

My answer

**Q22.** ISE291 Final Practice Questions, Q9

How many coefficients (parameters) do you need to estimate in a MISO linear regression model with 3 input numeric variables?

(A) 1 or 2

(B) 3 or 4

(C) 5 or 6

(D) 6 or 7

**Answer:** (B). Three slopes, plus the intercept if one is fitted (3 with `fit_intercept=False`, otherwise 4).

My answer

**Q23.** Quiz 3 T253 (Sec 04&05), Q7

For a MISO regression task with 3 input features, what will be the output of the following code (after the model is fitted)?

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression(fit_intercept=True)
A = reg.coef_[1]
print(A)
```

(A) The value of $\beta_0$

(B) The value of $\beta_1$

(C) The value of $\beta_2$

(D) The value of $\beta_3$

**Answer:** (C). `coef_` holds $[\beta_1, \beta_2, \beta_3]$, so index 1 is $\beta_2$; $\beta_0$ is in `intercept_`.

My answer

**Q24.** Final Exam T241, Q22

In MISO regression, which of the following Python functions is used to calculate the coefficient estimates using the Ordinary Least Squares (OLS) closed-form solution? The formula for the coefficient estimates is $\beta^* = (X^T X)^{-1} X^T y$, where $X$ is the matrix of input features starting with a column of ones and $y$ is the target variable.

(A) `np.dot(X, y)`

(B) `np.dot(X, X.T) @ np.linalg.inv(X)`

(C) `np.linalg.inv(X.T @ X) @ X.T @ y`

(D) `np.corrcoef(X, y)`

**Answer:** (C). Direct translation of $(X^T X)^{-1} X^T y$.

My answer

## Q25.

Final Exam T241, Q23

Given the following output for a MISO linear regression model:

```
2.0 [0.5 1.2]
```

Which code snippet will generate this output? (Assume X has 2 input variables and y is the output variable.)

(A)
```
model = LinearRegression()
model.fit(X, y)
print(model.intercept_, model.coef_)
```

(B)
```
model = Ridge(alpha=10)
model.fit(X, y)
print(model.coef_)
```

(C)
```
model = Lasso(alpha=10)
model.fit(X, y)
print(model.intercept_)
```

(D)
```
model = LinearRegression()
model.fit(X, y)
print(model.intercept_, model.coef_[0])
```

**Answer:** (A). Prints the scalar intercept followed by the array of the two slopes.

My answer

## Q26.

P7 (Final practice), Q11

Assume a dataframe `df` with 100 rows and the columns $x_1, x_2, x_3, x_4$, and $y$, where $y$ is the dependent variable. All other variables are independent variables. Assume $x_3$ is a categorical variable. Which of the following code will build a regression model that depicts the relationship between the dependent variable and all the numerical independent variables?

(A)
```
reg = LinearRegression()
reg.fit(df.drop(['x3','y'],axis=1).values, df['y'].values)
```

(B)
```
reg = LinearRegression()
reg.fit(df.drop(['y'],axis=1).values, df['y'].values)
```

(C)
```
reg = LinearRegression()
reg.fit(df.iloc[:,0:-1].values, df.iloc[:,-1].values)
```

(D)
```
reg.fit(df.iloc[:,0:-1].values, df.iloc[:,-1].values)
reg = LinearRegression()
```

**Answer:** (A). Both the output y and the categorical x3 must be removed from the inputs.

My answer

### Q27.

P8 (Quiz 03), Q12

To overcome the overfitting caused by OLS regression, we use

(A) Multiple Input Single Output (MISO) Linear Regression

(B) Single Input Single Output (SISO) Linear regression

(C) Penalized Linear Regression (PLR)

(D) Normalized Linear Regression (NLR)

**Answer:** (C). Ridge and Lasso add a penalty on large coefficients to control over-fitting.

My answer

### Q28.

Quiz 3 T253 (Sec 04&05), Q3

Which of the following represents the objective function for Ridge regression?

(A) $\frac{1}{n}\sum_{i=1}^{n}\left[h(x^i)-y_i\right]^2+\alpha\sum_{j=1}^{p}|\beta_j|$

(B) $\frac{1}{n}\sum_{i=1}^{n}\left[h(x^i)-y_i\right]^2$

(C) $\frac{1}{n}\sum_{i=1}^{n}\left[h(x^i)-y_i\right]^2+\alpha\sum_{j=1}^{p}\beta_j^2$

(D) $(X^TX+\alpha I)^{-1}X^Ty$

**Answer:** (C). Ridge = MSE + $\alpha\times$ sum of squared coefficients (A is Lasso, B is OLS).

My answer

### Q29.

P3 (Final practice), Q2

The following linear regression method results in the shrinking of the coefficients of the linear regression model without setting some of them to 0.

(A) Ridge Regression

(B) Lasso Regression

(C) Gradient Descent Regression

(D) Ordinary Least Squares Regression

**Answer:** (A). Ridge shrinks every coefficient but keeps all inputs; Lasso sets some exactly to zero.

My answer

**Q30.** F08 Quiz 3, Q11

Why is Lasso regression often used when dealing with many input features?

(A) It always produces the highest prediction accuracy.

(B) It performs feature selection by eliminating less important features.

(C) It increases the number of model parameters.

(D) It guarantees a perfect fit to the training data.

**Answer:** (B). The L1 penalty drives some coefficients exactly to zero, dropping those inputs.

My answer

**Q31.** ISE291 Final Practice Questions, Q18

In Ridge regression, as the value of the regularization parameter ($\alpha$) increases, the magnitude of the regression coefficients

(A) Typically decreases towards zero

(B) Typically increases towards infinity

(C) Typically stays the same

(D) No specific trend is generally applicable

**Answer:** (A). A stronger penalty pulls the coefficients towards zero ($\alpha = 0$ gives back OLS).

My answer

**Q32.** ISE291 Final Practice Questions, Q11

When input variables are uncorrelated, and all the input variables are related to the output variable, then the best method for linear regression is:

(A) Ridge

(B) LASSO

(C) OLS

(D) A and B

**Answer:** (C). Ridge helps with correlated inputs and Lasso with unrelated inputs; with neither problem, OLS is suitable.

My answer

**Q33.** P1 (Quiz 3 T251), Q7

Which of the following statements is FALSE?

(A) Lasso Regression applies only to SISO.

(B) Lasso does feature selection; it is useful when there are unrelated input columns.

(C) The aim of Cross-validation is to improve the model's generalizability.

(D) Ridge regression shrinks the estimate values; it is useful when there are correlated input columns.

**Answer:** (A). Lasso (like Ridge) is used with many inputs (MISO), e.g. the six-input real-estate case study.

My answer

**Q34.** P5 (Final practice), Q9

You can discover Overfitting when the model has

(A) great results in training and poor results in testing.

(B) poor results in both training and testing.

(C) great results in both training and testing.

(D) poor results in training and great results in testing.

**Answer:** (A). The coefficients fit the training data but do not generalize to new data.

My answer

**Q35.** Final Exam T241, Q38

What is the right code for splitting the data into train and test sets with a ratio of 75%:25%, respectively?

(A)
```
X_train, y_train, X_test, y_test =
    train_test_split(X, y, test_size=0.75, random_state=25)
```

(B)
```
X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.75, random_state=25)
```

(C)
```
X_train, y_train, X_test, y_test =
    train_test_split(X, y, test_size=0.25, random_state=25)
```

(D)
```
X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.25, random_state=25)
```

**Answer:** (D). Return order is X_train, X_test, y_train, y_test, and the test share is 0.25. (In every option `X = df.iloc[:,:-1].values`, `y = df.iloc[:,-1].values`.)

My answer

**Q36.** ISE291 Final Practice Questions, Q25

You are training a model using 5-fold cross-validation on a dataset with 500 samples. How many times will the model be trained and evaluated?

(A) 5 times, each on a different 80% vs 20% split

(B) 5 times, each on a different 70% vs 30% split

(C) 1 time using all the data

(D) 5 times using 5 bootstrapped subsets

**Answer:** (A). Each of the 5 equal folds (100 samples) is the validation set once; the other 400 train.

My answer

**Q37.** Quiz 3 Practice (ISE-291-13), Q6

Which of the following is the correct code to apply the z-score transformation on training and testing data?

(A)
```
scaler = StandardScaler()
scaler.fit(np.c_[X_train,y_train])
A_train = scaler.transform(np.c_[X_train,y_train])
A_test = scaler.transform(np.c_[X_test,y_test])
```

(B)
```
scaler = StandardScaler()
scaler.fit(np.c_[X_test,y_test])
A_train = scaler.transform(np.c_[X_train,y_train])
A_test = scaler.transform(np.c_[X_test,y_test])
```

(C)
```
scaler = MinMaxScaler()
scaler.fit(np.c_[X_train,y_train])
A_train = scaler.transform(np.c_[X_train,y_train])
A_test = scaler.transform(np.c_[X_test,y_test])
```

(D)
```
scaler = MinMaxScaler()
scaler.fit(np.c_[X_test,y_test])
A_train = scaler.transform(np.c_[X_train,y_train])
A_test = scaler.transform(np.c_[X_test,y_test])
```

**Answer:** (A). z-score = `StandardScaler`, and the scaler is fitted on the training data only, then applied to both sets.

My answer

**Q38.** P1 (Quiz 3 T251), Q9

Given the lines of code below, which of the following can provide the value of the best penalty coefficient?

```
from sklearn.linear_model import RidgeCV
reg = RidgeCV(alphas=[1e-4, 1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3],
              fit_intercept=False, cv=10).fit(X_train, y_train)
```

(A) `reg.intercept_`

(B) `reg.coef_`

(C) `reg.penalty_`

(D) `reg.alpha_`

**Answer:** (D). After cross-validation, the chosen $\alpha$ is stored in `alpha_`.

My answer

### Q39.

P1 (Quiz 3 T251), Q5

Which of the following methods provides a value that can be used to determine the best among OLS, Ridge, and Lasso models of a given dataset?

(A) `accuracy_score(y_test, y_pred)`

(B) `reg.coef_`

(C) `mean_squared_error(y_test, y_pred)`

(D) `reg.fit(X, y)`

**Answer:** (C). Compare the test-set MSE of the three models; the lowest wins.

My answer

### Q40.

Quiz_3 (docx), Q1

Given a regression dataset that is split into X_train, Y_train, X_test, and Y_test, which of the following code is used to fit a Linear Regression model, predict the output of the testing data, then calculate the mean squared error of the predicted values Y_pred?

(A)
```
reg = LinearRegression().fit(X_train, Y_train)
Y_pred = reg.predict(X_test)
print('MSE:', mean_squared_error(X_test, Y_pred))
```

(B)
```
reg = LinearRegression().fit(X_test, Y_test)
Y_pred = reg.predict(X_test)
print('MSE:', mean_squared_error(Y_train, Y_pred))
```

(C)
```
reg = LinearRegression().fit(X_train, Y_train)
Y_pred = reg.predict(X_test)
print('MSE:', mean_squared_error(Y_test, Y_pred))
```

(D)
```
reg = LinearRegression().fit(X_train, Y_train)
Y_pred = reg.predict(Y_test)
print('MSE:', mean_squared_error(Y_test, Y_pred))
```

**Answer:** (C). Fit on training data, predict from X_test, compare with the true Y_test.

My answer

# Part B: Programming

## Q41. [new]

Data for 10 students: attitude before the exam and exam score.

```
import pandas as pd
df = pd.DataFrame({'Attitude': [65, 67, 68, 70, 71, 72, 72, 73, 73, 75],
                   'Score':    [129, 126, 143, 156, 161, 158, 168, 166, 182, 201]})
```

(a) Compute $\beta_1 = r\, sd_y/sd_x$ and $\beta_0 = \bar{y} - \beta_1 \bar{x}$ with pandas. (b) Verify them with `LinearRegression`. (c) Predict the scores for attitudes 78, 74, 68 and 69. (d) Print the residual of the student with attitude 68 and the MSE of the fitted line.

**Answer:**

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# (a) by the formula
r = df['Attitude'].corr(df['Score'])
b1 = r * df['Score'].std() / df['Attitude'].std()
b0 = df['Score'].mean() - b1 * df['Attitude'].mean()
print('r =', round(r, 4), ' b1 =', round(b1, 2), ' b0 =', round(b0, 2))

# (b) verify with sklearn
reg = LinearRegression(fit_intercept=True)
reg.fit(df[['Attitude']], df['Score'])
print('sklearn: b1 =', np.round(reg.coef_, 2), ' b0 =', np.round(reg.intercept_, 2))

# (c) predictions
x_new = pd.DataFrame({'Attitude': [78, 74, 68, 69]})
print(np.round(reg.predict(x_new), 2).tolist())

# (d) residual and MSE
y_hat = reg.predict(df[['Attitude']])
print('residual (68):', round(df['Score'][2] - y_hat[2], 2))
print('MSE:', round(np.mean((df['Score'] - y_hat) ** 2), 2))
# r = 0.9418, b1 = 6.93, b0 = -330.46
# [210.3, 182.57, 140.97, 147.91]; residual 143 - 140.97 = 2.03; MSE = 52.92
```

My answer

## Q42. [new]

Implement gradient descent for SISO linear regression on $x = [1, 2, 3]$, $y = [2, 3, 5]$. Start at $\beta_0 = \beta_1 = 0$ with step size $\lambda = 0.1$. In each iteration compute the errors $\hat{y} - y$, the gradients (slope: $2\times$ mean of error$\times x$; intercept: $2\times$ mean of error), and update $\beta \leftarrow \beta - \lambda \times$gradient. Stop when the MSE no longer falls. Print the first two iterations, the final coefficients, and compare with `LinearRegression`.

**Answer:**

```
import numpy as np
from sklearn.linear_model import LinearRegression

x = np.array([1, 2, 3])
y = np.array([2, 3, 5])
b0, b1, lam = 0.0, 0.0, 0.1
old_mse = np.mean((b0 + b1 * x - y) ** 2)        # 12.667
it = 0
while True:
    err = (b0 + b1 * x) - y
    g1 = 2 * np.mean(err * x)
    g0 = 2 * np.mean(err)
    nb0, nb1 = b0 - lam * g0, b1 - lam * g1
    new_mse = np.mean((nb0 + nb1 * x - y) ** 2)
    if new_mse >= old_mse:                        # error stopped falling
        break
    b0, b1, old_mse = nb0, nb1, new_mse
    it += 1
    if it <= 2:
        print(f'iter {it}: b0={b0:.3f}, b1={b1:.3f}, MSE={new_mse:.3f}')
print(f'stopped after {it} iterations: b0={b0:.3f}, b1={b1:.3f}, MSE={old_mse:.3f}')

reg = LinearRegression().fit(x.reshape(-1, 1), y)
print('LinearRegression:', round(reg.intercept_, 3), np.round(reg.coef_, 3))
# iter 1: b0=0.667, b1=1.533, MSE=0.216
# iter 2: b0=0.587, b1=1.369, MSE=0.067
# stopped after 598 iterations: b0=0.333, b1=1.500, MSE=0.056  (same as LinearRegression)
```

My answer

## Q43. Final 291, Part-2 B-1

Column ERP is the output variable and all other columns are the input variables (computer hardware data).

```
import pandas as pd
df = pd.DataFrame({
```

```
  'MYCT': [125, 29, 29, 29, 29, 26, 23, 23, 23, 23, 400, 400, 60, 50],
  'MMIN': [256, 8000, 8000, 8000, 8000, 8000, 16000, 16000, 16000, 32000, 1000, 512, 2000,
   4000],
  'MMAX': [6000, 32000, 32000, 32000, 16000, 32000, 32000, 32000, 64000, 64000, 3000, 3500,
   8000, 16000],
  'CACH': [256, 32, 32, 32, 32, 64, 64, 64, 64, 128, 0, 4, 65, 65],
  'PRP':  [198, 269, 220, 172, 132, 318, 367, 489, 636, 1144, 38, 40, 92, 138],
  'ERP':  [199, 253, 253, 253, 132, 290, 381, 381, 749, 1238, 23, 24, 70, 117]})
```

Calculate and print the coefficient estimates using the closed form (the formula) of OLS regression for the first 12 records. Then predict and print the ERP for the input values [40, 4000, 16000, 32, 150].

**Answer:**

```
import numpy as np

d = df.iloc[:12]                          # first 12 records
Xo = d.iloc[:, 0:-1].values
y = d.iloc[:, -1].values
X = np.c_[np.ones(len(y)), Xo]            # column of ones for the intercept

best_beta = np.linalg.inv(X.T @ X) @ X.T @ y
print('The closed form estimates are:', np.round(best_beta, 4).tolist())

x_new = np.array([1, 40, 4000, 16000, 32, 150])   # leading 1 multiplies beta_0
print('Predicted ERP:', round(x_new @ best_beta, 2))
# [-245.408, 0.5475, 0.0129, 0.0082, 1.0143, 0.3145]
# Predicted ERP: 39.41
```

My answer

## Q44.

Homework 02 T213, B-5 and B-6

Run this cell to create the student data `dfB`; `Final` is the output and all other columns are inputs.

```
import numpy as np, pandas as pd
rng = np.random.default_rng(213); n = 40
hours = rng.integers(2, 20, n); exams = rng.integers(0, 10, n); absn = rng.integers(0, 12, n
    )
mid = np.clip(40 + 2.0*hours + rng.normal(0, 6, n), 0, 100).round(1)
proj = np.clip(50 + 1.5*hours + 1.0*exams + rng.normal(0, 8, n), 0, 100).round(1)
final = (5 + 1.2*hours + 0.8*exams - 1.5*absn + 0.35*mid + 0.2*proj + rng.normal(0, 4, n)).
    round(1)
dfB = pd.DataFrame({'Hours': hours, 'Exams': exams, 'Absences': absn,
                    'Mid': mid, 'Project': proj, 'Final': final})
```

(a) Use the formula $\theta = (X^T X)^{-1} X^T Y$ to calculate the OLS coefficient estimates. (b) Use sklearn to calculate the OLS estimates and report the largest difference between the two sets of coefficients.

**Answer:**

```
from sklearn.linear_model import LinearRegression

Xo = dfB.drop(columns=['Final']).values
y = dfB['Final'].values
X = np.c_[np.ones(len(y)), Xo]

# (a) closed form
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print('Closed form:', np.round(theta, 4).tolist())

# (b) sklearn
lr = LinearRegression().fit(Xo, y)
sk = np.r_[lr.intercept_, lr.coef_]
print('sklearn    :', np.round(sk, 4).tolist())
print('max |difference| =', np.max(np.abs(theta - sk)))
# both: [4.1677, 0.973, 0.8164, -1.5741, 0.327, 0.2809]; difference ~1e-13 (rounding only)
```

My answer

## Q45.

Homework 02 T213, B-7

Using the sklearn OLS model fitted on `dfB` (Q44), calculate the MSE for the new data `dfC`:

```
dfC = pd.DataFrame({'Hours': [5, 12, 18, 9, 15], 'Exams': [2, 6, 8, 1, 4],
                    'Absences': [10, 3, 0, 7, 2], 'Mid': [52, 66, 80, 55, 71],
                    'Project': [60, 75, 88, 58, 80], 'Final': [26, 55, 75, 33, 59]})
```

**Answer:**

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

lr = LinearRegression().fit(dfB.drop(columns=['Final']).values, dfB['Final'].values)
y_pred = lr.predict(dfC.drop(columns=['Final']).values)
print('Predictions:', np.round(y_pred, 2).tolist())
print('MSE on dfC:', round(mean_squared_error(dfC['Final'], y_pred), 3))
# Predictions: [28.79, 58.67, 79.1, 37.0, 64.57]
# MSE on dfC: 17.021
```

My answer

**Q46.** Homework 02 T213, B-8 and B-9

Taking all of `dfB` (Q44) as training data and `dfC` (Q45) as test data: (a) do the Ridge analysis with 10-fold cross-validation, picking the best $\alpha$ from 1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3, and report the MSE on `dfC`; (b) do the same with Lasso; (c) print both coefficient vectors.

**Answer:**

```
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error

X_b, y_b = dfB.drop(columns=['Final']).values, dfB['Final'].values
X_c, y_c = dfC.drop(columns=['Final']).values, dfC['Final'].values
alphas = [1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3]

ridge = RidgeCV(alphas=alphas, cv=10).fit(X_b, y_b)
print('Ridge: best alpha =', ridge.alpha_, ' MSE on dfC =',
      round(mean_squared_error(y_c, ridge.predict(X_c)), 3))

lasso = LassoCV(alphas=alphas, cv=10, random_state=0).fit(X_b, y_b)
print('Lasso: best alpha =', lasso.alpha_, ' MSE on dfC =',
      round(mean_squared_error(y_c, lasso.predict(X_c)), 3))

print('Ridge coef:', np.round(ridge.coef_, 3).tolist())
print('Lasso coef:', np.round(lasso.coef_, 3).tolist())
# Both pick alpha = 0.001 (MSE 17.021 and 17.023): almost no penalty is needed,
# so the coefficients are practically the OLS ones.
```

My answer

**Q47.** Quiz_3 (ipynb)

Run this cell to create the CPU data (ERP is the output; all other columns are inputs).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(7); n = 60
myct = rng.integers(20, 500, n); mmin = rng.choice([512, 1000, 2000, 4000, 8000, 16000], n)
mmax = mmin * rng.choice([2, 4, 8], n); cach = rng.choice([0, 8, 32, 64, 128], n)
chmin = rng.integers(0, 16, n); chmax = chmin + rng.integers(0, 32, n)
prp = (0.004*mmax + 0.9*cach + 2*chmax - 0.1*myct + rng.normal(0, 25, n)).round()
erp = (0.002*mmin + 0.003*mmax + 0.6*cach + 0.5*prp + rng.normal(0, 15, n)).round()
df = pd.DataFrame({'MYCT': myct, 'MMIN': mmin, 'MMAX': mmax, 'CACH': cach,
                   'CHMIN': chmin, 'CHMAX': chmax, 'PRP': prp, 'ERP': erp})
```

Use the first 40 rows for training and the rest for testing. Standardize (z-score) inputs and output using the training data, then fit OLS, RidgeCV and LassoCV (no intercept, alphas 1e-3 . . . 1e3, 15-fold CV, `random_state=0` for Lasso). Print each test MSE and state which model is best.

**Answer:**

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error

X_train = df.iloc[:40, :-1].values
y_train = df.iloc[:40, -1].values
X_test = df.iloc[40:, :-1].values
y_test = df.iloc[40:, -1].values

scaler = StandardScaler()
scaler.fit(np.c_[X_train, y_train])
A_train = scaler.transform(np.c_[X_train, y_train])
X_train, y_train = A_train[:, :-1], A_train[:, -1]
A_test = scaler.transform(np.c_[X_test, y_test])
X_test, y_test = A_test[:, :-1], A_test[:, -1]

alphas = [1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3]
reg1 = LinearRegression(fit_intercept=False).fit(X_train, y_train)
reg2 = RidgeCV(alphas=alphas, fit_intercept=False, cv=15).fit(X_train, y_train)
reg3 = LassoCV(alphas=alphas, fit_intercept=False, cv=15, random_state=0).fit(X_train,
    y_train)

mse = {name: mean_squared_error(y_test, r.predict(X_test))
       for name, r in [('OLS', reg1), ('Ridge', reg2), ('Lasso', reg3)]}
```

```
for k, v in mse.items():
    print(f'The MSE using {k} is: {v:.4f}')
print('Best model:', min(mse, key=mse.get))
# OLS 0.0172, Ridge 0.0165, Lasso 0.0152 -> Lasso is best
```

My answer

## Q48.

Quiz_03 (ipynb), B.1

Using the CPU data `df` from Q47, conduct regression analysis (Ridge and Lasso) using `test_size=0.4` (`random_state=0`), Ridge alphas = [5, 10, 50] and Lasso alphas = [10, 15, 50] (10-fold CV, no intercept, z-scored data). Comment on which technique gives the best result in mean squared error.

**Answer:**

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=0)

scaler = StandardScaler().fit(np.c_[X_train, y_train])
A_train = scaler.transform(np.c_[X_train, y_train])
X_train, y_train = A_train[:, :-1], A_train[:, -1]
A_test = scaler.transform(np.c_[X_test, y_test])
X_test, y_test = A_test[:, :-1], A_test[:, -1]

reg1 = RidgeCV(alphas=[5, 10, 50], fit_intercept=False, cv=10).fit(X_train, y_train)
reg2 = LassoCV(alphas=[10, 15, 50], fit_intercept=False, cv=10, random_state=0).fit(X_train,
    y_train)
m1 = mean_squared_error(y_test, reg1.predict(X_test))
m2 = mean_squared_error(y_test, reg2.predict(X_test))
print('The MSE using Ridge is:', round(m1, 4), ' alpha =', reg1.alpha_)
print('The MSE using Lasso is:', round(m2, 4), ' alpha =', reg2.alpha_)
print('Lasso coefficients:', reg2.coef_)
# Ridge 0.0187 (alpha 5), Lasso 0.6325: Ridge is best.
# On z-scored data alphas of 10 or more make Lasso set every coefficient to 0,
# so it predicts 0 for all test points.
```

My answer

**Q49.** Final 291, Part-2 B-2

Run this cell to create `dfB` (`price` is the output; all other columns are inputs).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(291); n = 120
carat = rng.uniform(0.2, 2.0, n).round(2); depth = rng.normal(61.5, 1.2, n).round(1)
table = rng.normal(57, 2, n).round(0); x = (6.4*carat**(1/3) + rng.normal(0, 0.05, n)).round
    (2)
y = (x + rng.normal(0, 0.03, n)).round(2); z = (0.618*x + rng.normal(0, 0.03, n)).round(2)
price = (4000*carat + 30*(table-57) - 20*(depth-61.5) + 300*(x-5.5) + rng.normal(0, 400, n))
    .round(0)
dfB = pd.DataFrame({'carat': carat, 'depth': depth, 'table': table,
                    'x': x, 'y': y, 'z': z, 'price': price})
```

Decide and print which regression method (Ridge or LASSO) is the best to be used here. Use 78% of the data for training and the rest for testing, MSE of the testing data as the decision criterion, possible alpha values [0.01, 0.02, 0.04, 0.08], 5-fold CV, and `random_state=291`.

**Answer:**

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error

X = dfB.drop(columns=['price']).values
y = dfB['price'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.78, random_state=291)

alphas = [0.01, 0.02, 0.04, 0.08]
ridge = RidgeCV(alphas=alphas, cv=5).fit(X_train, y_train)
lasso = LassoCV(alphas=alphas, cv=5, random_state=291, max_iter=100000).fit(X_train, y_train
    )
m_r = mean_squared_error(y_test, ridge.predict(X_test))
m_l = mean_squared_error(y_test, lasso.predict(X_test))
print(f'Ridge: alpha={ridge.alpha_}, test MSE={m_r:.1f}')
print(f'Lasso: alpha={lasso.alpha_}, test MSE={m_l:.1f}')
print('Best method:', 'Ridge' if m_r < m_l else 'LASSO')
# Ridge 114624.0, Lasso 111012.6 -> LASSO
```

My answer

**Q50.** Quiz 3 (ISE-291-03) ipynb, 2.2

Run this cell to create `data` (`Scores` is the last column and the output).

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42); n = 100
data = pd.DataFrame({'Age': rng.integers(18, 26, n), 'Selected': rng.integers(0, 2, n),
                     'Scholarship': rng.integers(5000, 10000, n), 'Gender': rng.integers(0,
    2, n),
                     'High_Marks': rng.integers(200, 1200, n)})
data['Scores'] = (0.08*data['High_Marks'] + 5*data['Selected'] + rng.normal(0, 6, n)).round
    ()
```

Split with `test_size=0.3, random_state=42`, z-score the data with the training set, then fit RidgeCV and LassoCV (no intercept, alphas 1e-3 . . . 1e3, 10-fold CV, `random_state=42` for Lasso). Print both test MSEs, the best Lasso $\alpha$, the Lasso coefficients, and the inputs Lasso dropped.

**Answer:**

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler().fit(np.c_[X_train, y_train])
A_train = scaler.transform(np.c_[X_train, y_train])
X_train, y_train = A_train[:, :-1], A_train[:, -1]
A_test = scaler.transform(np.c_[X_test, y_test])
X_test, y_test = A_test[:, :-1], A_test[:, -1]

alphas = [1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3]
reg2 = RidgeCV(alphas=alphas, fit_intercept=False, cv=10).fit(X_train, y_train)
reg3 = LassoCV(alphas=alphas, fit_intercept=False, cv=10, random_state=42).fit(X_train,
    y_train)
print('The MSE using Ridge is:', round(mean_squared_error(y_test, reg2.predict(X_test)), 4))
print('The MSE using Lasso is:', round(mean_squared_error(y_test, reg3.predict(X_test)), 4))
print('Best Lasso alpha:', reg3.alpha_)
coefs = pd.Series(reg3.coef_, index=data.columns[:-1])
print(coefs.round(4))
print('Inputs dropped by Lasso:', coefs[coefs.abs() < 1e-3].index.tolist())
```

```
# Ridge 0.0453, Lasso 0.0464, best alpha 0.01
# Lasso drops Scholarship and Gender (coefficients 0): they are unrelated to Scores.
```

My answer